

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Iulian Pavlov, FAF-241

Report

Laboratory work nr. 9

Watching System Calls with `strace`

Course: Operating Systems (Track A: the real OS, from the outside (Linux))

Checked by:
Reiţman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Contents

1 Theoretical Background and Objectives **1**
1.1 Laboratory Objectives and Scope 1
1.2 System Call Mechanics and File Descriptors 1

2 Part 1: Watching System Calls of a Real Command (`ls -l`) **2**
2.1 Tracing File and I/O Operations 2
2.2 System Call Summary Profiling (`strace -c ls -l`) 3
2.3 Answers to Questions (Observe – Part 1) 3

3 Part 2: Calling the Kernel Directly (`syscalls.c`) **4**
3.1 Implementation Details 4
3.2 Compilation and Execution Output 4
3.3 Tracing `./syscalls` with `strace` 5
3.4 Answers to Questions (Observe – Part 2) 6

4 Part 3: Counting One Kind of System Call **6**
4.1 Filtering and Counting Specific Syscalls 6
4.2 Analysis of Buffering and Output Redirection 6

5 Conclusions **7**

1 Theoretical Background and Objectives

1.1 Laboratory Objectives and Scope

Every interaction between an unprivileged user-space application and the Linux operating system kernel passes through the **system call interface**. Ordinary library routines execute entirely within the process's virtual address space, whereas system calls cross the hardware privilege boundary into kernel mode to perform file I/O, directory traversal, memory allocation, and process management.

The objectives of Laboratory Work Nr. 9 are:

- **Inspecting Real Commands with `strace`**: Tracing the file and I/O system calls (`openat`, `read`, `write`, `close`) executed by `ls -l`, and profiling its complete system call distribution using `strace -c`.
- **Direct Kernel I/O in C (`syscalls.c`)**: Writing, compiling, and executing a C program that performs file creation, writing, reading, and closing using raw POSIX system calls (`open`, `write`, `read`, `close`) instead of buffered `<stdio.h>` file streams.
- **Understanding File Descriptor Allocation**: Explaining why the first user-opened file receives descriptor `3` rather than `0`, `1`, or `2`.
- **Quantifying System Calls and Buffering**: Counting specific calls (`write`, `openat`) across commands and examining how `libc` stream buffering reduces kernel crossings.

1.2 System Call Mechanics and File Descriptors

1. **Tracing via `ptrace`**: The `strace` utility uses the Linux `ptrace()` system call to attach to a child process, intercepting each system call entry and exit to decode registers, arguments, and return codes (including negative `errno` errors such as `ENOENT`).
2. **Standard File Descriptors**: Every newly created POSIX process begins with three pre-opened descriptors inherited from its parent shell: `0` (`STDIN_FILENO`, standard input), `1` (`STDOUT_FILENO`, standard output), and `2` (`STDERR_FILENO`, standard error). The kernel always assigns the lowest unused integer descriptor when opening a new file.
3. **C Wrapper `open()` vs. Kernel `openat()`**: On modern Linux systems (such as x86_64 with glibc), the C function `open(path, flags, mode)` is implemented in user space as a thin wrapper around the `openat(AT_FDCWD, path, flags, mode)` kernel system call, where `AT_FDCWD` instructs the kernel to resolve relative paths against the current working directory.
4. **Direct Syscalls vs. `libc` Buffering**: Calling `write()` transfers bytes directly to the kernel on every invocation. By contrast, `printf()` writes into a user-space `stdout` buffer: connected to a terminal (TTY), `stdout` is *line-buffered* (flushing on each `\n`); redirected to a pipe, file, or `/dev/null`, it switches to *block buffering* and flushes multiple lines together.

2 Part 1: Watching System Calls of a Real Command (`ls -l`)

2.1 Tracing File and I/O Operations

We traced the open, read, write, and close system calls issued during a single execution of `ls -l`:

```
$ strace -f -e trace=openat,read,write,close ls -l
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/libcap.so.2", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\320{\0\0\0\0\0\0"..., 1024) =
    1024
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\300y\2\0\0\0\0\0"..., 1024) =
    1024
close(3)                                = 0
...
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
openat(AT_FDCWD, "/etc/nsswitch.conf", O_RDONLY|O_CLOEXEC) = 4
read(4, "# Name Service Switch configurat"..., 4096) = 372
read(4, "", 4096)                       = 0
close(4)                                = 0
openat(AT_FDCWD, "/etc/passwd", O_RDONLY|O_CLOEXEC) = 4
read(4, "root:x:0:0::/root:/usr/bin/bash\n"..., 4096) = 1615
close(4)                                = 0
openat(AT_FDCWD, "/etc/group", O_RDONLY|O_CLOEXEC) = 4
read(4, "root:x:0:root\nsys:x:3:bin\nmem:x:"..., 4096) = 930
close(4)                                = 0
...
close(3)                                = 0
openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = 3
read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\17\0\0\0\17\0\0\0\0"..., 4096) =
    2120
read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\17\0\0\0\17\0\0\0\0"..., 4096) =
    1317
close(3)                                = 0
write(1, "total 96\n-rw-r--r-- 1 sunflower "..., 444) = 444
close(1)                                = 0
close(2)                                = 0
+++ exited with 0 +++
```

From the trace, `ls -l` proceeds through four clear stages:

1. **Dynamic Linking**: `ld-linux` opens `/etc/ld.so.cache`, `libcap.so.2`, and `libc.so.6` to map shared libraries into memory.
2. **Locale and Directory Opening**: After probing `coreutils.mo` localization files, `ls` opens directory `.` with `O_DIRECTORY` (receiving descriptor `3`).
3. **Metadata and Identity Resolution**: To display owner names, group names, and local timestamps, `ls` consults `/etc/nsswitch.conf`, `/etc/passwd`, `/etc/group`, and `/etc/localtime`.

4. **Output and Cleanup**: `ls` emits its formatted listing via `write(1, ...)` and closes standard streams `1` and `2` before exiting.

2.2 System Call Summary Profiling (`strace -c ls -l`)

We ran `strace -c ls -l >/dev/null` to obtain the aggregate count of all system calls invoked during one execution:

```
$ strace -c ls -l >/dev/null
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 31.16    0.000592           8        68        29 openat
 20.32    0.000386         386         1           execve
 12.68    0.000241          11        21           mmap
  8.53    0.000162           3        41           close
  7.11    0.000135           4        33           statx
  4.11    0.000078          13         6           mprotect
  2.47    0.000047           2        16           fstat
  2.32    0.000044           6         7           llistxattr
  2.05    0.000039           3        13           read
  1.79    0.000034           3         9           newfstatat
  1.42    0.000027           4         6         4 prctl
  0.95    0.000018           2         8           lseek
  0.79    0.000015           5         3           brk
  0.79    0.000015           3         4           rt_sigprocmask
  0.47    0.000009           4         2         2 ioctl
  0.47    0.000009           4         2           getdents64
  0.47    0.000009           4         2           getrandom
  0.42    0.000008           8         1         1 access
  0.37    0.000007           3         2           futex
  0.26    0.000005           5         1           write
  0.26    0.000005           5         1           prlimit64
  0.21    0.000004           4         1           arch_prctl
  0.21    0.000004           4         1           set_tid_address
  0.21    0.000004           4         1           set_robust_list
  0.16    0.000003           3         1           rseq
------ ----------- ----------- --------- --------- ----------------
100.00    0.001900           7       251        36 total
```

In the initial host desktop session (where the `systemd-userdbd` socket was reachable), `strace -c ls -l >/dev/null` recorded **260** total system calls (**30** `openat`, **28** `close`, **15** `read`, **1** `write`, plus socket/epoll IPC calls). In the isolated verification environment where `/run/systemd/userdb/` is unreachable, `libnss_systemd` falls back to scanning filesystem `userdb` paths, yielding **251** total system calls (**123** filtered `openat`/`read`/`write`/`close` calls). Thus, one simple `ls -l` consistently executes roughly **250–260** system calls.

2.3 Answers to Questions (Observe – Part 1)

1. **Which system call does `ls` use to print to the screen? To which descriptor does it write?**

   - **System call**: `write` (specifically `write(1, ...)`).
   - **File descriptor**: `1` (`STDOUT_FILENO`, standard output). On an interactive terminal, `ls` issues one `write(1, ...)` call per output line; when redirected

or piped, `libc` block-buffers `stdout` and emits the full listing in one `write(1, ...)` call.

3 Part 2: Calling the Kernel Directly (`syscalls.c`)

3.1 Implementation Details

We implemented `syscalls.c` using the POSIX system calls `open()`, `write()`, `read()`, and `close()` directly to create `note.txt`, write `"hello, kernel\n"`, reopen the file read-only, and read the stored bytes back into user memory:

Listing 1: Implementation of syscalls.c using POSIX system calls directly

```c
/* syscalls.c - open, write, read and close, using only system calls.
 * Build:  gcc -Wall -Wextra -o syscalls syscalls.c
 * Run:    ./syscalls
 * See the calls: strace ./syscalls
 */
#include <fcntl.h>   /* open */
#include <unistd.h>  /* write, read, close */
#include <string.h>  /* strlen */
#include <stdio.h>   /* printf */

int main(void) {
    const char *msg = "hello, kernel\n";

    /* open() is a system call: ask the kernel for a file, get back a descriptor */
    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    printf("open returned descriptor %d\n", fd);

    /* write() is a system call: hand the bytes to the kernel */
    ssize_t n = write(fd, msg, strlen(msg));
    if (n < 0) { perror("write"); close(fd); return 1; }
    printf("write sent %zd bytes\n", n);
    close(fd);

    /* read it back, through the kernel again */
    fd = open("note.txt", O_RDONLY);
    if (fd < 0) { perror("open"); return 1; }
    char buf[64];
    n = read(fd, buf, sizeof(buf) - 1);
    if (n < 0) { perror("read"); close(fd); return 1; }
    buf[n] = '\0';
    printf("read got %zd bytes: %s", n, buf);
    close(fd);

    return 0;
}
```

3.2 Compilation and Execution Output

Compiling with `-Wall -Wextra` succeeded with zero warnings, and executing `./syscalls` produced the expected output and a 14-byte `note.txt` file:

```
$ gcc -Wall -Wextra -o syscalls syscalls.c
$ ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
$ wc -c note.txt
14 note.txt
```

3.3 Tracing `./syscalls` with `strace`

Tracing `openat`, `write`, `read`, and `close` during execution reveals both the dynamic loader setup and the exact six file-I/O calls from `main()`:

```
$ strace -e trace=openat,write,read,close ./syscalls
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\300y\2\0\0\0\0\0"..., 1024) =
    1024
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(1, "open returned descriptor 3\n", 27) = 27
write(3, "hello, kernel\n", 14)         = 14
write(1, "write sent 14 bytes\n", 20)   = 20
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
write(1, "read got 14 bytes: hello, kernel"..., 33) = 33
close(3)                                = 0
+++ exited with 0 +++
```

Table 1 summarizes the direct mapping between each C call in `syscalls.c` and the observed Linux kernel system call.

C Source Statement	Observed Kernel System Call	Return
`open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644)`	`openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644)`	3
`write(fd, msg, strlen(msg))`	`write(3, "hello, kernel\n", 14)`	14
`close(fd)`	`close(3)`	0
`open("note.txt", O_RDONLY)`	`openat(AT_FDCWD, "note.txt", O_RDONLY)`	3
`read(fd, buf, sizeof(buf) - 1)`	`read(3, "hello, kernel\n", 63)`	14
`close(fd)`	`close(3)`	0

Table 1: Mapping between C library calls in `syscalls.c` and kernel system calls observed via `strace`.

Running `strace -c ./syscalls >/dev/null` confirms that the entire program executes only **41** total system calls across 18 distinct syscall types from startup to exit.

3.4 Answers to Questions (Observe – Part 2)

1. **The descriptor returned by the first `openat` is 3, not 0, 1 or 2. Why are those three already taken?**
   When any Linux/UNIX process is created and started by a shell, three standard file descriptors are already open by convention:

   - `0` (`STDIN_FILENO`): Standard input (`stdin`), attached to the terminal keyboard or input stream.
   - `1` (`STDOUT_FILENO`): Standard output (`stdout`), attached to the terminal screen or redirected output target.
   - `2` (`STDERR_FILENO`): Standard error (`stderr`), attached to the terminal screen for diagnostics.

   Because POSIX mandates that `open()`/`openat()` return the **lowest-numbered unused descriptor**, the first newly opened file (`note.txt`) receives descriptor `3`. After `close(3)` frees slot 3, the second `open("note.txt", O_RDONLY)` immediately reuses descriptor `3`.

4 Part 3: Counting One Kind of System Call

4.1 Filtering and Counting Specific Syscalls

We isolated and counted individual system calls (`write` and `openat`) for both `ls -l` and `./syscalls`:

```
$ strace -f -e trace=write ls -l 2>&1 >/dev/null | grep -c '^write'
1

$ strace -f -e trace=openat ls -l 2>&1 >/dev/null | grep -c '^openat'
68

$ strace -e trace=write ./syscalls 2>&1 >/dev/null | grep -c '^write'
2
```

4.2 Analysis of Buffering and Output Redirection

The command pipeline `2>&1 >/dev/null | grep -c '^write'` performs two subtle operations:

1. **Separating Program Output from `strace` Output**: Because `strace` prints trace lines to `stderr` (fd 2) while the traced program prints to `stdout` (fd 1), `2>&1` sends `strace`'s `stderr` into the pipe while `>/dev/null` discards the program's `stdout`. Without `>/dev/null`, `./syscalls`'s own output line `"write sent 14 bytes"` matches `^write` in `grep`, falsely inflating the count from 2 to 3.

2. **TTY Line Buffering vs. Redirected Block Buffering**: Redirecting `stdout` to `/dev/null` switches `libc`'s `stdout` stream from line buffering to block buffering:

- In `./syscalls` with `stdout` redirected to `/dev/null`, all three `printf()` messages accumulate in the user-space buffer and flush in **1** `write(1, ..., 80)` call on exit, plus **1** direct `write(3, "hello, kernel\n", 14)` call, totaling **2** `write` syscalls. On a live terminal (TTY), `stdout` flushes after every `\n`, producing **4** `write` syscalls (3 `write(1, ...)` + 1 `write(3, ...)`).
- Similarly, `ls -l` issues **1** `write(1, ...)` call when redirected to `/dev/null`, versus one `write(1, ...)` per line on a terminal.

Command	**Syscall**	**Redirected (`>/dev/null`)**	**TTY (`strace -o`)**	**Explanation**
`ls -l`	`write`	1	8 (header + 7 entries)	Block-buffered flush vs. per-line TTY output
`ls -l`	`openat`	68 / 30 (host)	68 / 30 (host)	Shared libs, locales, `.`, NSS/userdb, `/etc/localtime`
`./syscalls`	`write`	2	4	1 merged `printf` flush vs. 3 TTY flushes, plus 1 `write(3, ...)`

Table 2: Comparison of measured system call counts under output redirection and interactive terminal execution.

5 Conclusions

1. **Visibility across the System Call Boundary**: `strace` exposes every transition between user space and the Linux kernel, revealing that a routine `ls -l` invocation executes roughly 250–260 system calls spanning dynamic linking, localization, directory traversal, identity resolution, and terminal output.

2. **Direct POSIX File I/O**: Writing `syscalls.c` with `open()`, `write()`, `read()`, and `close()` confirmed how user-space C calls map directly to kernel syscalls (`openat`, `write`, `read`, `close`) without buffered `FILE*` wrappers.

3. **Reserved Descriptors and Reuse**: Descriptors `0`, `1`, and `2` are permanently reserved at startup for `stdin`, `stdout`, and `stderr`; consequently, the kernel assigns descriptor `3` to the first opened file and recycles slot `3` once closed.

4. **Buffering Efficiency**: Counting `write` calls under redirection versus interactive TTY execution demonstrated how `libc` stream buffering merges multiple `printf()` outputs into a single kernel transition, reducing system call overhead.